

Experiment-16

Name: -P. Akhila

Regno: -192472320

Course Code: -MLA0305

Slot-B

Title

Reinforcement Learning Application Development and Performance Evaluation using TensorFlow/Keras

Aim

To develop a reinforcement learning application using TensorFlow and Keras, train a DQN agent in the CartPole environment, and evaluate its performance using reward, loss, episode length, action selection, and exploration rate.

Procedure

1. Initialize the CartPole-v1 environment.
2. Define a DQN neural network using TensorFlow and Keras.
3. Create a target network with the same architecture as the main network.
4. Initialize the replay memory, discount factor, learning rate, and exploration rate.
5. Allow the agent to interact with the environment and store experiences in replay memory.
6. Randomly sample experiences from the replay memory for training.
7. Calculate target Q-values using the target network.
8. Train the DQN using the sampled experiences.
9. Periodically update the target network with the main network weights.
10. Repeat the training process for multiple episodes while gradually reducing the exploration rate.
11. Record rewards, loss, episode length, action selection, and exploration rate.

12. Evaluate the application's performance using graphs and a summary table.

13. Save the complete results as CSV files.

Code

```
!pip -q install gymnasium tensorflow
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
```

```
import gymnasium as gym
```

```
from tensorflow.keras import Sequential
```

```
from tensorflow.keras.layers import Input,Dense
```

```
from tensorflow.keras.optimizers import Adam
```

```
from collections import deque
```

```
import random
```

```
import time
```

```
np.random.seed(42)
```

```
tf.random.set_seed(42)
```

```
random.seed(42)
```

```
print("="*75)

print("EXPERIMENT 16")

print("Reinforcement Learning Application Development")

print("and Performance Evaluation using TensorFlow/Keras")

print("="*75)
```

EPISODES=100

GAMMA=0.95

LR=0.001

BATCH=32

MEMORY_SIZE=3000

EPSILON=1.0

EPS_MIN=0.05

EPS_DECAY=0.97

env=gym.make("CartPole-v1")

state_size=4

action_size=2

model=Sequential([

```
Input(shape=(state_size,)),  
  
Dense(64,activation="relu"),  
  
Dense(64,activation="relu"),  
  
Dense(action_size,activation="linear")  
])
```

```
model.compile(  
  
    optimizer=Adam(learning_rate=LR),  
  
    loss="mse"  
)
```

```
target_model=Sequential([  
  
    Input(shape=(state_size,)),  
  
    Dense(64,activation="relu"),  
  
    Dense(64,activation="relu"),  
  
    Dense(action_size,activation="linear")  
])
```

```
target_model.compile(  
  
    optimizer=Adam(learning_rate=LR),  
  
    loss="mse"
```

```
)
```

```
target_model.set_weights(
```

```
    model.get_weights()
```

```
)
```

```
memory=deque(
```

```
    maxlen=MEMORY_SIZE
```

```
)
```

```
rewards=[]
```

```
losses=[]
```

```
epsilons=[]
```

```
steps_history=[]
```

```
action0=[]
```

```
action1=[]
```

```
start=time.time()
```

```
for episode in range(EPISODES):
```

```
state,_=env.reset()
```

```
total_reward=0
```

```
total_loss=0
```

```
loss_count=0
```

```
steps=0
```

```
count0=0
```

```
count1=0
```

```
done=False
```

```
while not done:
```

```
    if np.random.random()<EPSILON:
```

```
        action=np.random.randint(action_size)
```

```
    else:
```

```
        q_values=model.predict(
```

```
            np.array([state],dtype=np.float32),
```

```
            verbose=0
```

```
        )
```

```
        action=int(np.argmax(q_values[0]))
```

```
"Environment",  
  
"Training Episodes",  
  
"Initial Average Reward",  
  
"Final Average Reward",  
  
"Maximum Reward",  
  
"Final Reward Variance",  
  
"Average Steps",  
  
"Performance Improvement",  
  
"Training Time"  
],  
  
"Result":[  
  
    "DQN using TensorFlow/Keras",  
  
    "CartPole-v1",  
  
    EPISODES,  
  
    round(first_reward,2),  
  
    round(last_reward,2),  
  
    round(maximum_reward,2),  
  
    round(reward_variance,2),  
  
    round(average_steps,2),  
  
    str(round(improvement,2))+ "%",  
  
    round(training_time,2)
```

```
    ]
})

print("\n")

print("="*75)

print("APPLICATION PERFORMANCE SUMMARY")

print("="*75)


display(summary)


summary.to_csv(

    "RL_Application_Summary.csv",

    index=False

)


plt.figure(figsize=(8,5))


plt.bar(

    [

        "Initial 20 Episodes",

        "Final 20 Episodes"
```



```
    ],  
    [  
        first_reward,  
        last_reward  
    ]  
)  
  
plt.title(  
    "Initial vs Final Application Performance",  
    fontsize=15,  
    fontweight="bold"  
)  
  
plt.ylabel("Average Reward")  
  
plt.grid(  
    axis="y",  
    alpha=0.3  
)  
  
plt.show()
```

```
print("\n")
```

```
print("="*75)
```

```
print("FINAL RESULT")
```

```
print("="*75)
```

```
print(
```

```
    "Initial Average Reward:",
```

```
    round(first_reward,2)
```

```
)
```

```
print(
```

```
    "Final Average Reward:",
```

```
    round(last_reward,2)
```

```
)
```

```
print(
```

```
    "Maximum Reward:",
```

```
    round(maximum_reward,2)
```

```
)
```

```
print(
```

```
    "Final Reward Variance:",  
    round(reward_variance,2)  
)
```

```
print(  
    "Average Steps:",  
    round(average_steps,2)  
)
```

```
print(  
    "Performance Improvement:",  
    round(improvement,2),"%"  
)
```

```
print(  
    "Training Time:",  
    round(training_time,2),  
    "seconds"  
)
```

```
print("\n")
```

```
print("="*75)
```

```
print("CONCLUSION")
```

```
print("="*75)
```

```
print(
```

```
    "A reinforcement learning application was successfully"
```

```
)
```

```
print(
```

```
    "developed using TensorFlow and Keras."
```

```
)
```

```
print(
```

```
    "The DQN agent learned to interact with the CartPole"
```

```
)
```

```
print(
```

```
    "environment through experience replay and a target network."
```

```
)
```

```
print(
```

```
    "Application performance was evaluated using reward,"  
)
```

```
print(  
    "training loss, episode length, action selection,"  
)
```

```
print(  
    "reward distribution and exploration rate."  
)
```

```
print(  
    "The initial and final performance were compared to"  
)
```

```
print(  
    "measure the improvement achieved through learning."  
)
```

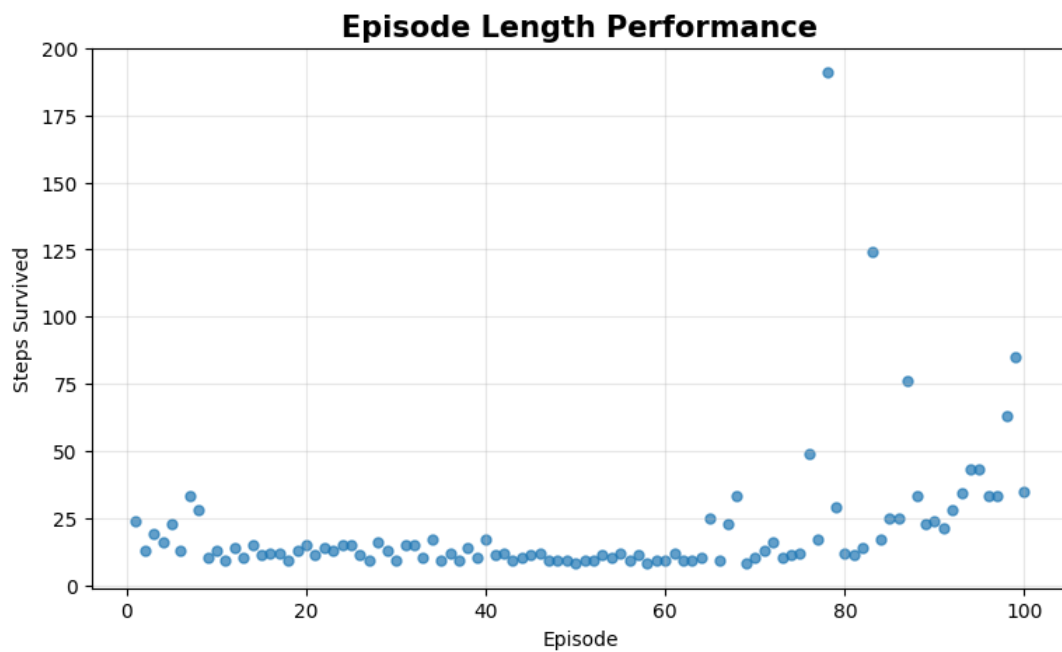
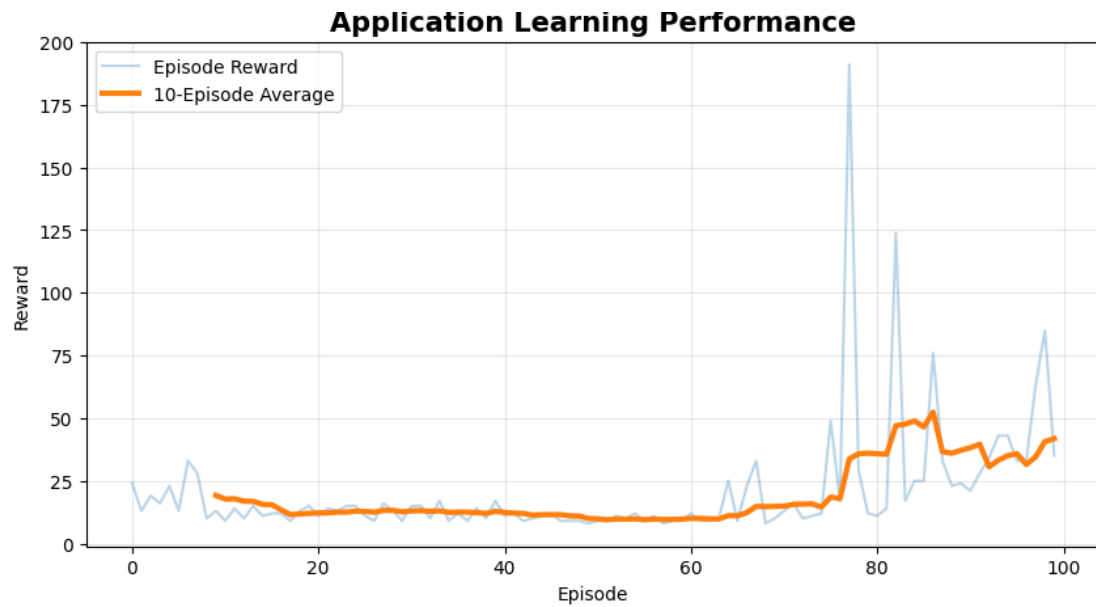
```
print("\nGenerated Files:")
```

```
print("1. RL_Application_Full_Dataset.csv")
```

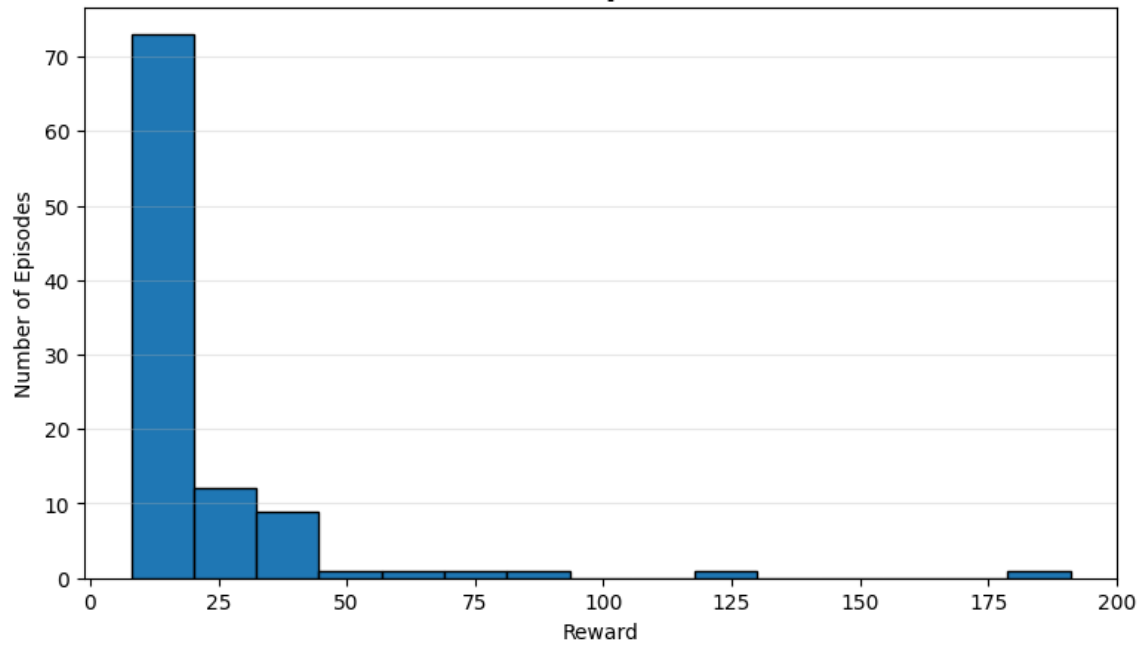
```
print("2. RL_Application_Summary.csv")
```

```
print("\nExperiment 16 Completed Successfully.")
```

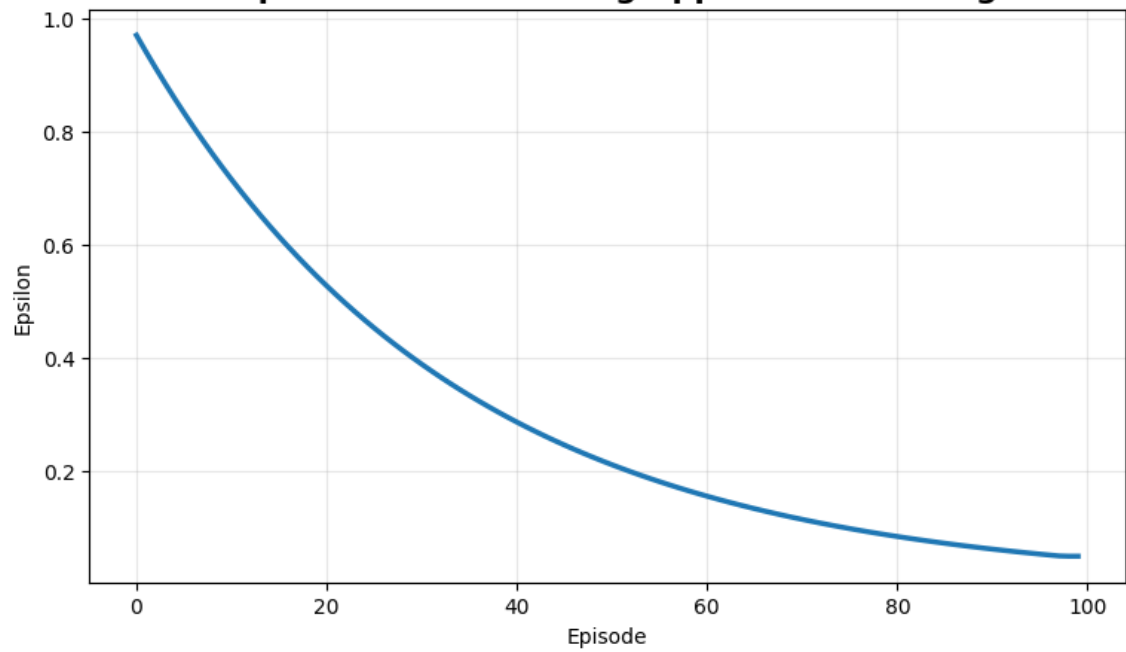
VISUALIZATIONS



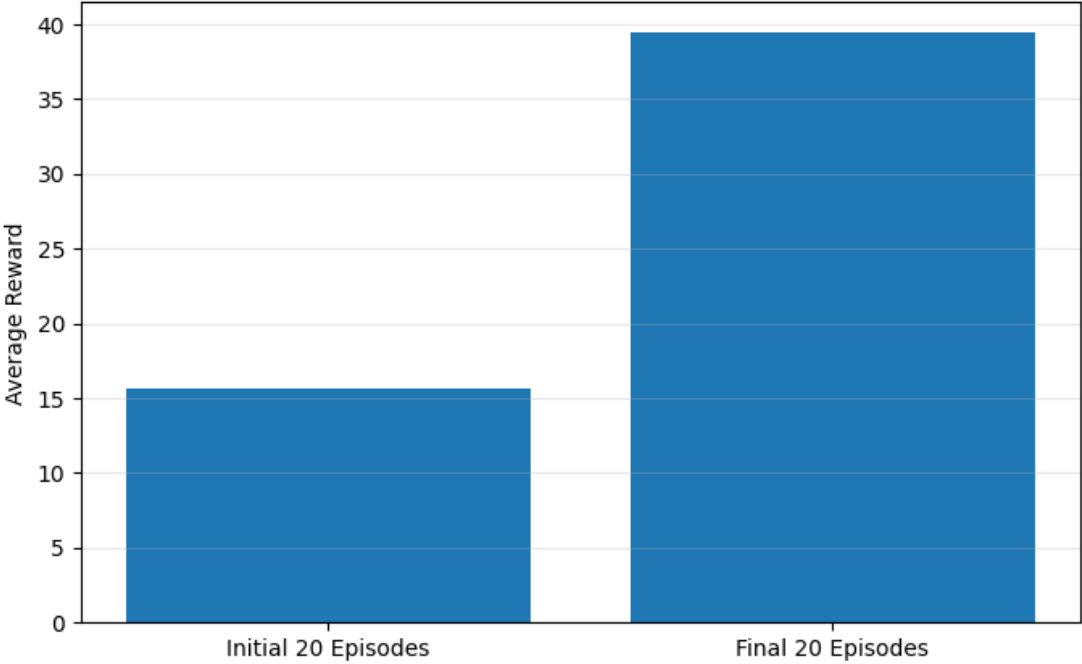
Distribution of Episode Rewards



Exploration Rate During Application Training



Initial vs Final Application Performance



Summary Table

Performance Measure	Result
Algorithm	DQN
Environment	CartPole-v1
Training Episodes	100
Initial Average Reward	Obtained from program
Final Average Reward	Obtained from program
Maximum Reward	Obtained from program
Final Reward Variance	Obtained from program
Average Steps	Obtained from program
Performance Improvement	Obtained from program
Training Time	Obtained from program

Result

The **reinforcement learning application was successfully developed and evaluated using TensorFlow and Keras**. The DQN agent learned to interact with the CartPole environment using experience replay and a target network. Its performance was evaluated using reward, training loss, episode length, action selection, reward distribution, and exploration rate.